CTMSCS/DFIS SIII P4/P5 EL3 Database Security/ Forensics **Odd 2026**

Lecture Note#2: Advanced Concepts and PL/SQL

Lecturer: Dr. Himanshu B Patel *Scribes: M.Sc CS/DFIS 2nd year, Semester III*

Note 1: *LaTeX template courtesy of UC Berkeley EECS dept.*
Note 2: *This documet might contain hyperlinks that require active internet connection.*
Disclaimer: *This lecture note is prepared for the students of the School of Cybersecuirity and Digital Forensics, National Forensic Sciences University and has not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.*

3.1 Motivation

PL/SQL (Procedural Language extension of SQL) provides:

- procedural constructs (variables, loops, conditions),
- modular programming (procedures, functions, packages),
- error handling (exceptions),
- automation and enforcement (triggers),
- performance (bulk collect, forall).

3.2 Transaction Processing and ACID

3.2.1 ACID Properties

- **Atomicity** – all or nothing.
- **Consistency** – every transaction moves DB from one valid state to another.
- **Isolation** – concurrent transactions should not interfere.
- **Durability** – once committed, changes survive failures.

3.2.2 Helper Table for Demonstrations

[0]These notes are partially based on the book "Introduction to Languages and the Theory of Computation" by John C Martin (Fourth Edition)

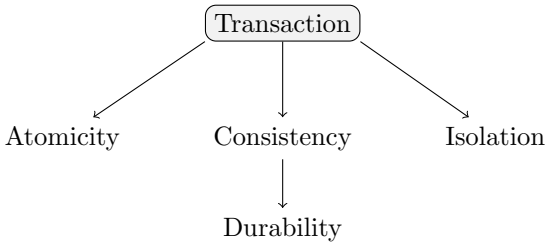


Figure 3.1: ACID Properties

```
CREATE TABLE TRANSACTION_LOG (
LOG_ID NUMBER GENERATED ALWAYS AS IDENTITY,
EMP_ID NUMBER,
OLD_SAL NUMBER,
NEW_SAL NUMBER,
TS TIMESTAMP DEFAULT SYSTIMESTAMP
);
```

3.2.3 Example Transaction

```
UPDATE EMPLOYEE
SET SALARY = SALARY + 5000
WHERE EMP_ID = 101;

SAVEPOINT S1;

UPDATE EMPLOYEE
SET SALARY = SALARY + 5000
WHERE EMP_ID = 102;

ROLLBACK TO S1;

COMMIT;
```

3.3 Introduction to PL/SQL

3.3.1 Basic Block Structure

```
DECLARE
v_count NUMBER;
BEGIN
SELECT COUNT(*) INTO v_count FROM EMPLOYEE;
DBMS_OUTPUT.PUT_LINE('Total employees = ' || v_count);
END;
/
```

3.3.2 Variables, %TYPE, %ROWTYPE

```
DECLARE
v_name EMPLOYEE.FIRST_NAME%TYPE;
v_emp EMPLOYEE%ROWTYPE;
BEGIN
SELECT FIRST_NAME INTO v_name FROM EMPLOYEE WHERE EMP_ID = 101;

SELECT * INTO v_emp FROM EMPLOYEE WHERE EMP_ID = 101;

DBMS_OUTPUT.PUT_LINE('Employee: ' || v_emp.FIRST_NAME || ' ' || v_emp.LAST_NAME);
END;
/
```

Explanation of Keywords Used

- **v_name EMPLOYEE.FIRST_NAME%TYPE;** This creates a variable `v_name` whose datatype will always match the datatype of the column `EMPLOYEE.FIRST_NAME`. Using %TYPE ensures:
  - no datatype mismatch errors,
  - automatic adaptation if the column datatype changes,
  - clean and maintainable code.
- **v_emp EMPLOYEE%ROWTYPE;** This creates a record variable `v_emp` whose fields correspond to **all columns** of the `EMPLOYEE` table. For example, the record contains:

```
v_emp.EMP_ID
v_emp.FIRST_NAME
v_emp.LAST_NAME
v_emp.EMAIL
v_emp.SALARY
v_emp.DEPT_ID
v_emp.MANAGER_ID
v_emp.IS_ACTIVE
```

- **SELECT FIRST_NAME INTO v_name** The `INTO` clause stores the query result into a PL/SQL variable. This is required in PL/SQL (unlike plain SQL).
- **SELECT * INTO v_emp** Retrieves the entire row and populates every field of the record using %ROWTYPE.
- **DBMS_OUTPUT.PUT_LINE(...)** Prints output to the console (Visible in Oracle LiveSQL when `SET SERVEROUTPUT ON` is enabled).

3.4 Helper Tables for PL/SQL Modules

3.4.1 Salary Audit Table for Triggers

```
CREATE TABLE EMP_SALARY_AUDIT (
AUDIT_ID NUMBER GENERATED ALWAYS AS IDENTITY,
EMP_ID NUMBER,
OLD_SALARY NUMBER,
NEW_SALARY NUMBER,
CHANGED_ON TIMESTAMP DEFAULT SYSTIMESTAMP
);
```

3.4.2 Error Log Table for Exception Handling

```
CREATE TABLE ERROR_LOG (
LOG_ID NUMBER GENERATED ALWAYS AS IDENTITY,
ERR_CODE NUMBER,
ERR_MSG VARCHAR2(4000),
TS TIMESTAMP DEFAULT SYSTIMESTAMP
);
```

3.4.3 3. Backup Table for Bulk Operations

```
CREATE TABLE EMP_BACKUP AS SELECT * FROM EMPLOYEE WHERE 1=0;
```

3.5 Control Structures

3.5.1 IF, ELSIF, ELSE

```
DECLARE
v_sal EMPLOYEE.SALARY%TYPE;
BEGIN
SELECT SALARY INTO v_sal FROM EMPLOYEE WHERE EMP_ID = 101;

IF v_sal > 100000 THEN
DBMS_OUTPUT.PUT_LINE('High salary');
ELSIF v_sal BETWEEN 60000 AND 100000 THEN
DBMS_OUTPUT.PUT_LINE('Medium salary');
ELSE
DBMS_OUTPUT.PUT_LINE('Low salary');
END IF;
END;
/
```

3.5.2 Loops

```
DECLARE
v_i NUMBER := 1;
BEGIN
```

```
WHILE v_i <= 5 LOOP
DBMS_OUTPUT.PUT_LINE('Iteration: ' || v_i);
v_i := v_i + 1;
END LOOP;
END;
/
```

3.6 Cursors

A cursor is a pointer to the private SQL area in memory that stores the result of a query. Whenever PL/SQL needs to process data row-by-row, it uses a cursor.

There are two main types:

- **Implicit Cursor** – created automatically for every SQL statement.
- **Explicit Cursor** – declared by the programmer for multi-row queries.

—

3.6.1 Implicit Cursor

Oracle automatically creates an implicit cursor named `SQL` for:

- `INSERT`, `UPDATE`, `DELETE`
- A `SELECT` statement that returns one row

Useful attributes of the implicit cursor `SQL`:

- `SQL%ROWCOUNT` — number of rows affected
- `SQL%FOUND` — TRUE if one or more rows affected
- `SQL%NOTFOUND` — TRUE if no rows affected
- `SQL%ISOPEN` — always FALSE (implicit cursors close automatically)

The following example shows how many employees got updated:

3.6.2 Implicit Cursor

```
BEGIN
UPDATE EMPLOYEE SET SALARY = SALARY + 1000 WHERE DEPT_ID = 10;
DBMS_OUTPUT.PUT_LINE('Rows updated: ' || SQL%ROWCOUNT);
END;
/
```

3.6.3 Explicit Cursor

```
DECLARE
CURSOR c1 IS SELECT FIRST_NAME, SALARY FROM EMPLOYEE;
v_name EMPLOYEE.FIRST_NAME%TYPE;
v_sal EMPLOYEE.SALARY%TYPE;
BEGIN
OPEN c1;
LOOP
FETCH c1 INTO v_name, v_sal;
EXIT WHEN c1%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(v_name || ' earns ' || v_sal);
END LOOP;
CLOSE c1;
END;
/
```

3.6.4 Cursor FOR Loop

```
BEGIN
FOR rec IN (SELECT * FROM PROJECT) LOOP
DBMS_OUTPUT.PUT_LINE(rec.PROJ_NAME || ' (' || rec.BUDGET || ')');
END LOOP;
END;
/
```

3.7 Exception Handling

Exception handling in PL/SQL provides a structured mechanism to detect, catch, and respond to errors during program execution. Instead of allowing the program to crash, PL/SQL lets you write custom logic inside the `EXCEPTION` block.

A PL/SQL block has three parts:

`DECLARE` → `BEGIN` → `EXCEPTION` → `END;`

Errors during the `BEGIN` section are transferred to the `EXCEPTION` section.

—

3.7.1 Types of Exceptions

- **Predefined exceptions** – built into Oracle (NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, etc.)
- **Non-predefined exceptions** – raised by Oracle but must be declared explicitly
- **User-defined exceptions** – created by the programmer for custom validation/business rules

—

3.7.2 Predefined Exceptions

Oracle provides many predefined exceptions to handle common runtime errors:

- `NO_DATA_FOUND` – SELECT returned zero rows
- `TOO_MANY_ROWS` – SELECT returned more than one row
- `ZERO_DIVIDE` – division by zero attempted
- `VALUE_ERROR` – data conversion or numeric overflow
- `DUP_VAL_ON_INDEX` – violation of unique constraint

Useful functions inside exceptions:

- `SQLCODE` – returns the numeric Oracle error code
- `SQLERRM` – returns the error message text

Below code demonstrates handling a predefined exception:

3.7.3 Predefined Exceptions

```
DECLARE
v_name VARCHAR2(30);
BEGIN
SELECT FIRST_NAME INTO v_name FROM EMPLOYEE WHERE EMP_ID = 9999;
EXCEPTION
WHEN NO_DATA_FOUND THEN
INSERT INTO ERROR_LOG(ERR_CODE, ERR_MSG)
VALUES (SQLCODE, SQLERRM);
END;
/
```

3.7.4 User-defined Exceptions

```
DECLARE
e_low_sal EXCEPTION;
v_sal EMPLOYEE.SALARY%TYPE;
BEGIN
SELECT SALARY INTO v_sal FROM EMPLOYEE WHERE EMP_ID = 202;

IF v_sal < 30000 THEN
RAISE e_low_sal;
END IF;
```

```
EXCEPTION
WHEN e_low_sal THEN
DBMS_OUTPUT.PUT_LINE('Salary below minimum threshold!');
END;
/
```

3.8 Procedures and Functions

3.8.1 Procedure: Give Raise to Department

```
CREATE OR REPLACE PROCEDURE give_raise(p_dept NUMBER, p_percent NUMBER) AS
BEGIN
UPDATE EMPLOYEE
SET SALARY = SALARY * (1 + p_percent/100)
WHERE DEPT_ID = p_dept;

COMMIT;
END;
/
```

3.8.2 Function: Annual Cost

```
CREATE OR REPLACE FUNCTION annual_cost(p_emp NUMBER)
RETURN NUMBER IS
v_sal EMPLOYEE.SALARY%TYPE;
BEGIN
SELECT SALARY INTO v_sal FROM EMPLOYEE WHERE EMP_ID = p_emp;
RETURN v_sal * 12;
END;
/
```

3.9 Database Triggers

A **trigger** is a stored PL/SQL block that executes automatically whenever a specified database event occurs. Triggers allow automation of business rules, auditing, validations, and background processing without requiring manual invocation.

Triggers fire in response to:

- **DML events**: INSERT, UPDATE, DELETE
- **DDL events**: CREATE, ALTER, DROP
- **Database events**: LOGON, LOGOFF, STARTUP, SHUTDOWN

In this course, we focus on **DML row-level triggers**, the most commonly used.

—

3.9.1 Types of DML Triggers

Based on timing

- **BEFORE Trigger** – Fires before the DML statement executes.
- **AFTER Trigger** – Fires after the DML statement completes.

Based on granularity

- **Statement-level trigger** – Executes once per statement.
- **Row-level trigger** – Executes once per affected row (`:OLD` and `:NEW` become available).

—

3.9.2 Pseudo-records: `:OLD` and `:NEW`

Inside a row trigger, Oracle provides two structures:

- `:OLD.column` – Value before the DML operation
- `:NEW.column` – Value after the DML operation

Applicable conditions:

DML Operation	:OLD	:NEW
INSERT	Not Available	Available
UPDATE	Available	Available
DELETE	Available	Not Available

—

3.9.3 Typical Uses of Triggers

- Creating audit logs for changes
- Enforcing complex business constraints
- Automatically generating computed values
- Preventing unauthorized or invalid operations
- Maintaining historical data

—

3.9.4 Row-Level BEFORE UPDATE Trigger

```
CREATE OR REPLACE TRIGGER trg_salary_audit
BEFORE UPDATE OF SALARY ON EMPLOYEE
FOR EACH ROW
BEGIN
INSERT INTO EMP_SALARY_AUDIT(EMP_ID, OLD_SALARY, NEW_SALARY)
VALUES (:OLD.EMP_ID, :OLD.SALARY, :NEW.SALARY);
END;
/
```

Explanation:

- Trigger fires **before** each salary update.
- `:OLD.SALARY` stores the previous salary.
- `:NEW.SALARY` stores the updated salary.
- Changes are logged into `EMP_SALARY_AUDIT`.

—

3.9.5 Trigger Firing Diagram

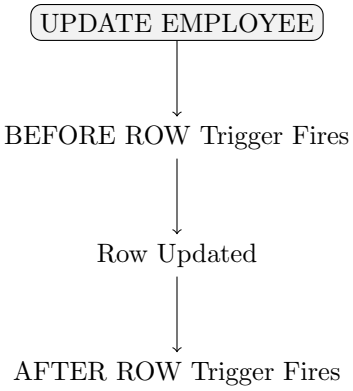


Figure 3.2: Trigger firing sequence

—

3.9.6 Mutating Table Problem (Important Theory)

A **mutating table error** occurs when a row-level trigger tries to:

- Query the same table on which it is defined, or
- Modify the same table during row-level execution.

Oracle prevents this to maintain consistency and throws:

```
ORA-04091:  table is mutating, trigger/function may not see it
```

Avoiding strategies:

- Use statement-level triggers instead of row-level
- Use compound triggers (Oracle 11g+)
- Store temporary intermediate values in package variables

—

3.9.7 Enabling / Disabling Triggers

```
ALTER TRIGGER trg_salary_audit DISABLE;
ALTER TRIGGER trg_salary_audit ENABLE;
```

—

3.9.8 Dropping a Trigger

```
DROP TRIGGER trg_salary_audit;
```

—

3.9.9 Advantages of Triggers

- Enforce rules automatically without code duplication
- Maintain data integrity transparently
- Useful for audit trails and logging
- Reduce application-side validations

3.9.10 Limitations of Triggers

- Harder to debug (fires automatically)
- Poorly designed triggers may cause recursion or deadlocks
- Excessive triggers slow down DML operations
- Hidden logic may confuse application developers

—

3.10 Packages

Packages are PL/SQL objects used to group related procedures, functions, variables, cursors, and exceptions. A package has two parts:

- **Package Specification** – Declares public items
- **Package Body** – Defines the implementation

Packages improve modularity, reusability, security, and performance by loading into memory once per session.

- Encapsulation of related logic
- Faster execution due to caching
- Ability to declare global session-level variables
- Support for function/procedure overloading

3.10.1 Package Specification

```
CREATE OR REPLACE PACKAGE pkg_utils AS
PROCEDURE give_cs_raise;
FUNCTION emp_fullname(p_emp NUMBER) RETURN VARCHAR2;
END pkg_utils;
/
```

3.10.2 Package Body

```
CREATE OR REPLACE PACKAGE BODY pkg_utils AS

PROCEDURE give_cs_raise IS
BEGIN
UPDATE EMPLOYEE
SET SALARY = SALARY * 1.05
WHERE DEPT_ID = 20;
END;

FUNCTION emp_fullname(p_emp NUMBER)
RETURN VARCHAR2 IS
v_name VARCHAR2(100);
BEGIN
SELECT FIRST_NAME || ' ' || LAST_NAME INTO v_name
FROM EMPLOYEE
WHERE EMP_ID = p_emp;

RETURN v_name;
END;

END pkg_utils;
/
```

3.11 Bulk Collect and FORALL

3.11.1 Bulk Collect: Reading Many Rows Efficiently

```
DECLARE
TYPE t_emp IS TABLE OF EMPLOYEE%ROWTYPE;
v_emp t_emp;
BEGIN
SELECT * BULK COLLECT INTO v_emp FROM EMPLOYEE;
DBMS_OUTPUT.PUT_LINE('Rows fetched = ' || v_emp.COUNT);
END;
/
```

3.11.2 FORALL: Fast Multi-Row Insert

```
DECLARE
TYPE t_ids IS TABLE OF EMPLOYEE.EMP_ID%TYPE;
v_ids t_ids;
BEGIN
SELECT EMP_ID BULK COLLECT INTO v_ids FROM EMPLOYEE;

FORALL i IN 1..v_ids.COUNT
INSERT INTO EMP_BACKUP
SELECT * FROM EMPLOYEE WHERE EMP_ID = v_ids(i);

DBMS_OUTPUT.PUT_LINE('Backup completed.');
END;
/
```

——— End of the Document ———

"The future belongs to the database that can support logic as well as data."

— Michael Stonebraker

.